HALLOWEEN PROPS · BUZZ WIRE · BOARD V1 · ESP32 DEVKIT V1 + DY-SV17F

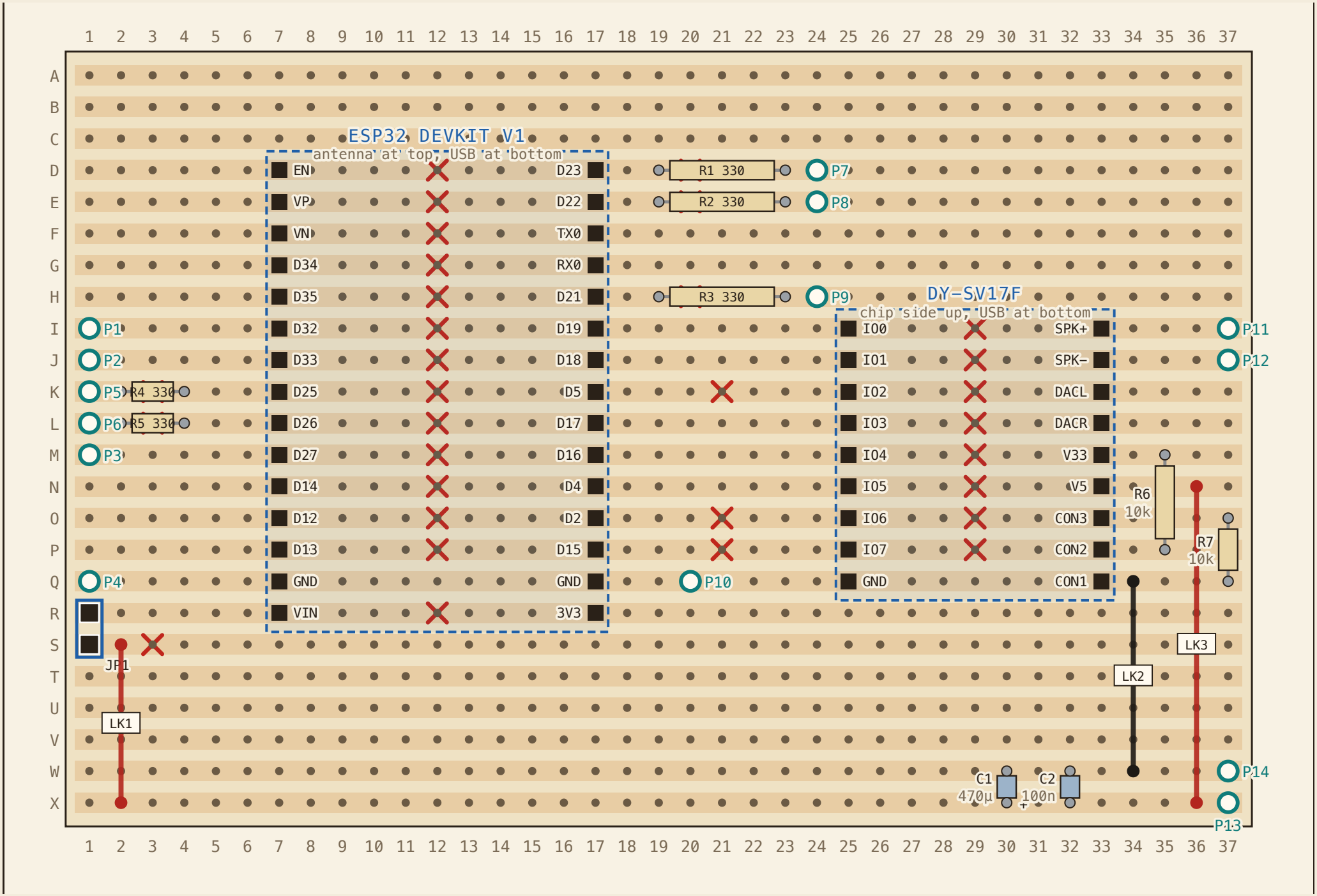
Buzz Wire Board

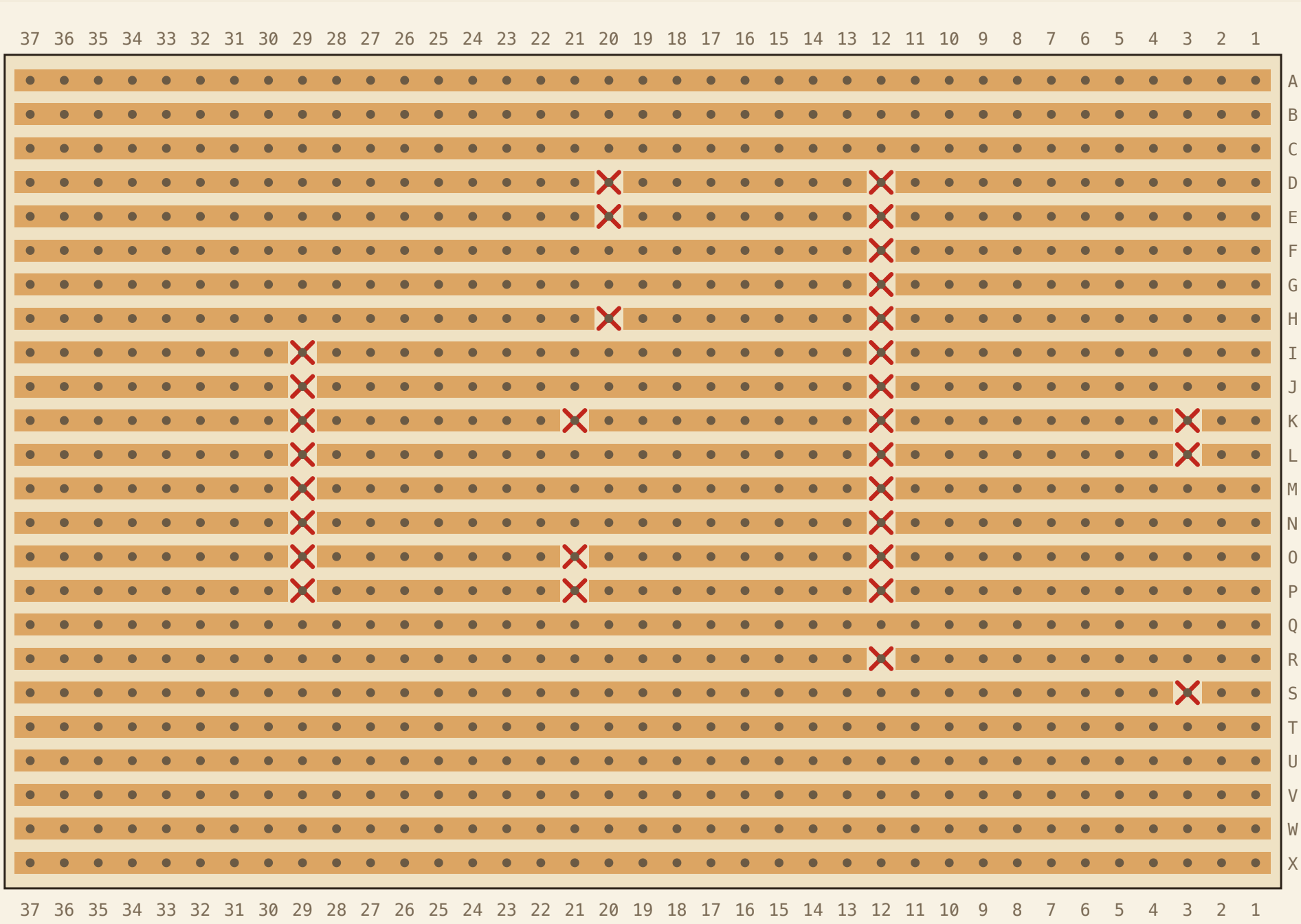
Touch the start pad, then guide the wand to the finish pad without touching the wire. You get **3 lives**: each touch **boos** and puts out a red LED, losing all 3 brings an **evil laugh**, and reaching the finish brings a **cheer**. Everything runs on **5V only**, on the same **24-strip × 37-hole** stripboard with the strips running lengthways.

Checked: the netlist passed the connection check: 88 pins, 31 cuts, 3 links, no shorts and no broken connections. Removing a cut or a link makes the check fail, so it is catching mistakes. The sketch hasn't been compiled yet, so treat it as untested until your first upload.

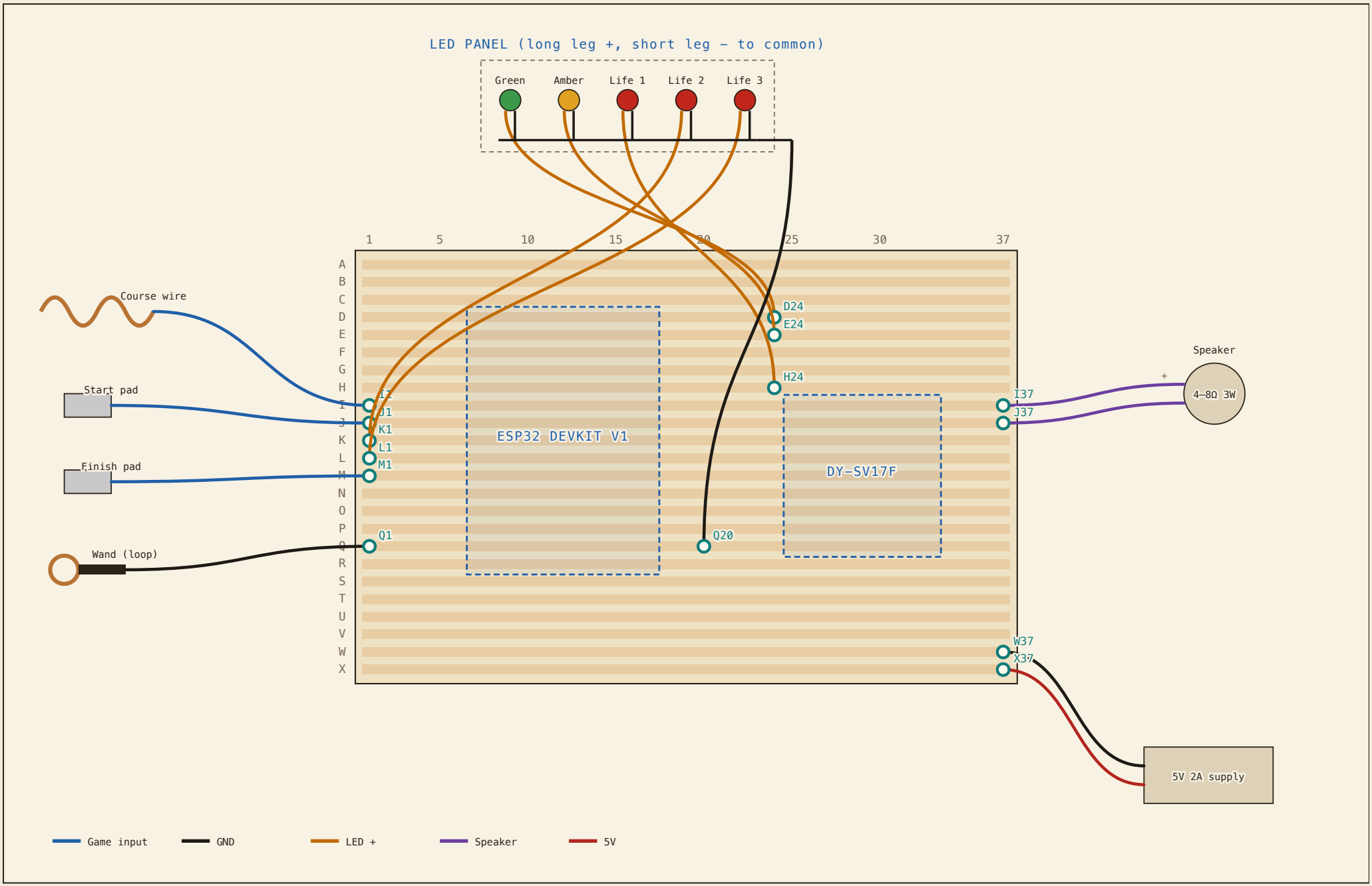
Three checks before you solder:

1. **The sound module's orientation.** With the **chip side up and the USB socket towards the bottom**, the **IO0…GND** row must be on the **left** (column 25), with IO0 at the top. If it's on the right, stop and tell me: the layout would need mirroring.
2. **The ESP32's left-hand labels** should read, top to bottom, **EN, VP, VN, D34, D35, D32, D33, D25, D26, D27, D14, D12, D13, GND, VIN**.
3. The sound module has **bare pads**. Solder a 9-pin male header strip into each row of half-holes, then plug it into 9-pin female headers on the board. Put the ESP32 in 15-pin female headers too.


ESP32 DEVKIT V1
antenna at top, USB at bottom
EN
VP
VN
D34
D35
D32
D33
D25
D26
D27
D14
D12
D13
GND
VIN
D23
D22
TX0
RX0
D21
D19
D18
D5
D17
D16
D4
D2
D15
GND
3V3
R1 330
R2 330
R3 330
R4 330
R5 330
P1
P2
P3
P4
P5
P6
P7
P8
P9
P10
P11
P12
P13
P14
DY-SV17F
chip side up, USB at bottom
IO0
IO1
IO2
IO3
IO4
IO5
IO6
IO7
GND
SPK+
SPK-
DACL
DACR
V33
V5
CON3
CON2
CON1
R6
10k
R7
10k
C1
470µ
C2
100n
JP1
LK1
LK2
LK3



Copper side, flipped left to right. Drill out each X with a 3.5 mm bit. 31 cuts.


LED PANEL (long leg +, short leg − to common)
Green
Amber
Life 1
Life 2
Life 3
1
5
10
15
20
25
30
37
A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
P
Q
R
S
T
U
V
W
X
Course wire
Start pad
Finish pad
Wand (loop)
I1
J1
K1
L1
M1
Q1
D24
E24
H24
Q20
ESP32 DEVKIT V1
DY-SV17F
I37
J37
Speaker
+
4–8Ω 3W
W37
X37
5V 2A supply
Game input
GND
LED +
Speaker
5V

1. Off-board wires (the wiring diagram as a table)

PAD	HOLE	GOES TO	WIRE
P1	I1	Course wire, at one end	blue
P2	J1	Start pad (metal plate)	blue
P3	M1	Finish pad (metal plate)	blue
P4	Q1	Wand loop, through flexible wire	black
P7	D24	Green LED + (long leg)	orange
P8	E24	Amber LED +	orange
P9	H24	Life 1 red LED +	orange
P5	K1	Life 2 red LED +	orange
P6	L1	Life 3 red LED +	orange
P10	Q20	All five LED − (short legs) joined together	black
P11	I37	Speaker +	purple
P12	J37	Speaker −	purple
P13	X37	5V supply +, rated 2A or more	red
P14	W37	5V supply −	black

4. Components

PART	HOLES	NOTE
ESP32 DevKit V1	D7–R7, D17–R17	2 × 15-way female header, antenna towards row A
DY-SV17F	I25–Q25, I33–Q33	2 × 9-way female header, chip side up, USB towards the bottom
JP1	R1, S1	2-way header + jumper cap. **Remove it while USB is plugged into the ESP32**
R1–R3 330Ω	D19→D23, E19→E23, H19→H23	Green, amber and life 1 LEDs, each across its cut
R4, R5 330Ω	K2→K4, L2→L4	Life 2 and 3 LEDs. Stand them upright across the cut
R6 10kΩ	M35→P35	CON2 to the module's 3.3V output (V33). Stand it upright
R7 10kΩ	O37→Q37	CON3 to GND. Stand it upright
C1 470µF 10V+	X30 (+), W30 (−)	The **stripe marks the − leg**, which goes in W30
C2 100nF	X32, W32	Ceramic, either way round

Sound module mode: **I/O Independent 0**. CON1 goes to GND (strip Q), CON2 goes high through R6, and CON3 goes to GND through R7. In this mode, pulling an IO pin low plays the matching file to the end. Strips A–C and T–V stay empty.

2. Track cuts (31)

COLUMN	STRIPS	WHY
12	D to P, and R (14)	Splits the ESP32's two pin rows. **Don't cut Q**: it's GND on both sides
3	K, L (2)	R4 and R5 bridge these cuts
20	D, E, H (3)	R1, R2 and R3 bridge these cuts
21	K, O, P (3)	Keeps D5, D2 and D15 off the sound module, because they misbehave at boot
29	I to P (8)	Splits the sound module's two pin rows. Leave Q joined: CON1 goes to GND
3	S (1)	JP1 strip

3. Wire links (3)

LINK	HOLES	DOES
LK1	S2 → X2	5V rail → JP1
LK2	Q34 → W34	GND strip Q → GND rail W
LK3	N36 → X36	5V rail → sound module V5

5. Sound files

FILE	MODULE PIN	ESP32	SOUND
00001.mp3	IO0	D19	Boo
00002.mp3	IO1	D18	Cheer
00003.mp3	IO2	not connected	Filler: a copy of the boo
00004.mp3	IO3	D17	Evil laugh (game over)
00005.mp3	IO4	D16	"Come and play…"

Copy the files to the module's drive **in number order**, into the top folder. Then run `dot_clean /Volumes/<drive name>` and eject. **Unplug the module's USB cable before you play the game**. IO5 (D4) is wired up as a spare for `00006.mp3`.

6. Power-up checks, in order

1. **Power off, no modules fitted, meter on continuity:** X37 to W37 must **not** beep. W37 to Q1 must beep (the ground link). X37 to N33 must beep (5V reaches the module's V5 socket).
2. **5V on, still no modules:** X37 reads 5.0V and N33 reads 5.0V. With JP1 fitted, hole R7 (the ESP32 VIN socket) reads 5.0V. Every other socket hole reads 0V.
3. **5V off, fit the ESP32, JP1 off, USB in:** upload the sketch. You should see "Buzz wire ready" and every LED lights for 1 second.
4. **Game test with no sound yet:** touch the wand to the start pad and the amber and three red LEDs come on. Touch the wire and one red goes out, with "Touch!" in Serial Monitor.

5. **USB out, 5V off. Fit the sound module with its files loaded, fit JP1, then 5V on:** play a full game. You should hear the boo, the laugh and the cheer.

7. The sketch

Save it as `buzz-wire/buzz_wire/buzz_wire.ino`. In Arduino, select the board **DOIT ESP32 DEVKIT V1**. It needs no libraries. You can change the grace time after a touch (1.5 s), the game timeout and the attract interval at the top of the sketch. Each touch that lasts more than 1.5 s costs another life, so resting the wand on the wire doesn't pay.

```
// Buzz Wire v1 (ESP32 DevKit V1 + DY-SV17F in I/O Independent mode 0)
// Touch the start pad to begin, get to the finish pad without touching the wire.
// 3 lives: each touch boos and puts out a red LED. Lose all 3: evil laugh. Reach the finish: cheer.
// Board: "DOIT ESP32 DEVKIT V1" (or "ESP32 Dev Module"). No libraries needed.

// ---------- Inputs (wand is GND, touching pulls these LOW) ----------
const byte WIRE_PIN   = 32;
const byte START_PIN  = 33;
const byte FINISH_PIN = 27;

// ---------- LEDs (330R to each anode, cathodes to GND) ----------
const byte GREEN_LED = 23;
const byte AMBER_LED = 22;
const byte LIFE_LED[3] = { 21, 25, 26 };
const byte HEARTBEAT_LED = 2;          // Blue LED on the ESP32 board

// ---------- DY-SV17F triggers (pull LOW briefly to play) ----------
const byte SND_BOO     = 19;           // IO0 -> 00001.mp3
const byte SND_CHEER   = 18;           // IO1 -> 00002.mp3
const byte SND_LAUGH   = 17;           // IO3 -> 00004.mp3
const byte SND_ATTRACT = 16;           // IO4 -> 00005.mp3

// ---------- Tuning ----------
const byte LIVES = 3;
const unsigned long GRACE_MS     = 1500;   // After a touch, ignore the wire this long
const unsigned long DEBOUNCE_MS  = 8;      // Touch must last this long to count
const unsigned long GAME_TIMEOUT = 90000;  // Abandoned game returns to waiting
const unsigned long ATTRACT_MIN  = 60000;  // "Come and play" every 1–3 minutes while waiting
const unsigned long ATTRACT_MAX  = 180000;

enum State { WAITING, PLAYING, WON, LOST };
State state = WAITING;
byte lives = LIVES;
unsigned long stateSince = 0, lastHit = 0, lastActivity = 0, nextAttract = 0;

void playSound(byte pin) {
  pinMode(pin, OUTPUT);
  digitalWrite(pin, LOW);
  delay(100);
```

```
  pinMode(pin, INPUT);                // Release: the module's own pull-up takes it high
}

bool touching(byte pin) {             // LOW for DEBOUNCE_MS = a real touch
  if (digitalRead(pin) == HIGH) return false;
  delay(DEBOUNCE_MS);
  return digitalRead(pin) == LOW;
}

void showLives() {
  for (byte i = 0; i < LIVES; i++) digitalWrite(LIFE_LED[i], i < lives ? HIGH : LOW);
}

void allOff() {
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(AMBER_LED, LOW);
  for (byte i = 0; i < LIVES; i++) digitalWrite(LIFE_LED[i], LOW);
}

void enter(State s) {
  state = s;
  stateSince = millis();
  allOff();
  switch (s) {
    case WAITING:
      Serial.println(F("Waiting: touch the start pad"));
      nextAttract = millis() + random(ATTRACT_MIN, ATTRACT_MAX);
      break;
    case PLAYING:
      Serial.println(F("Go!"));
      lives = LIVES;
      lastHit = lastActivity = millis();
      digitalWrite(AMBER_LED, HIGH);
      showLives();
      break;
    case WON:
      Serial.print(F("Finished with "));
      Serial.print(lives);
      Serial.println(F(" lives left"));
      playSound(SND_CHEER);
```

```
      break;
    case LOST:
      Serial.println(F("Game over"));
      playSound(SND_LAUGH);
      break;
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(WIRE_PIN, INPUT_PULLUP);
  pinMode(START_PIN, INPUT_PULLUP);
  pinMode(FINISH_PIN, INPUT_PULLUP);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(AMBER_LED, OUTPUT);
  for (byte i = 0; i < LIVES; i++) pinMode(LIFE_LED[i], OUTPUT);
  pinMode(HEARTBEAT_LED, OUTPUT);
  pinMode(SND_BOO, INPUT);             // Left as inputs so nothing plays at power-up
  pinMode(SND_CHEER, INPUT);
  pinMode(SND_LAUGH, INPUT);
  pinMode(SND_ATTRACT, INPUT);
  randomSeed(esp_random());

  // Lamp test: every LED on for 1 s
  digitalWrite(GREEN_LED, HIGH);
  digitalWrite(AMBER_LED, HIGH);
  for (byte i = 0; i < LIVES; i++) digitalWrite(LIFE_LED[i], HIGH);
  delay(1000);
  Serial.println(F("Buzz wire ready"));
  enter(WAITING);
}

void loop() {
  unsigned long now = millis();
  digitalWrite(HEARTBEAT_LED, (now % 2000) < 50);

  switch (state) {
    case WAITING:
      digitalWrite(GREEN_LED, (now % 2000) < 1000);   // Slow green blink
      if (touching(START_PIN)) enter(PLAYING);
```

```
      else if ((long)(now - nextAttract) >= 0) {
        playSound(SND_ATTRACT);
        nextAttract = now + random(ATTRACT_MIN, ATTRACT_MAX);
      }
      break;

    case PLAYING:
      if (touching(FINISH_PIN)) { enter(WON); break; }
      if (now - lastHit > GRACE_MS && touching(WIRE_PIN)) {
        lastHit = lastActivity = now;
        lives--;
        Serial.print(F("Touch! Lives left: "));
        Serial.println(lives);
        if (lives == 0) { enter(LOST); break; }
        playSound(SND_BOO);
        for (byte f = 0; f < 3; f++) {              // Flash the remaining lives
          for (byte i = 0; i < LIVES; i++) digitalWrite(LIFE_LED[i], LOW);
          delay(80);
          showLives();
          delay(80);
        }
      }
      if (touching(START_PIN)) lastActivity = now;   // Back at the start: still playing
      if (now - lastActivity > GAME_TIMEOUT) enter(WAITING);
      break;

    case WON:                                        // Green and amber flash for 5 s
      digitalWrite(GREEN_LED, (now / 150) % 2);
      digitalWrite(AMBER_LED, !((now / 150) % 2));
      if (now - stateSince > 5000) enter(WAITING);
      break;

    case LOST:                                       // All reds flash for 5 s
      for (byte i = 0; i < LIVES; i++) digitalWrite(LIFE_LED[i], (now / 120) % 2);
      if (now - stateSince > 5000) enter(WAITING);
      break;
  }
  delay(2);
}
```